

# Table of Contents

|                                                             |          |
|-------------------------------------------------------------|----------|
| <b>ChimeraTK ApplicationCore — Tutorial</b>                 | <b>2</b> |
| Table of Contents . . . . .                                 | 2        |
| 1. What is ApplicationCore? . . . . .                       | 2        |
| 2. Project setup from scratch . . . . .                     | 3        |
| 2.1 Required dependencies . . . . .                         | 3        |
| 2.2 Directory layout . . . . .                              | 3        |
| 2.3 CMakeLists.txt . . . . .                                | 3        |
| 2.4 main.cc . . . . .                                       | 4        |
| 3. Core concepts . . . . .                                  | 4        |
| 3.1 Class hierarchy . . . . .                               | 4        |
| 3.2 Application . . . . .                                   | 4        |
| 3.3 ApplicationModule . . . . .                             | 5        |
| 3.4 ModuleGroup and VariableGroup . . . . .                 | 5        |
| 4. Accessor types . . . . .                                 | 6        |
| 4.1 Scalar accessors . . . . .                              | 6        |
| 4.2 Array accessors . . . . .                               | 6        |
| 4.3 Void accessors (triggers) . . . . .                     | 7        |
| 4.4 Reading and writing . . . . .                           | 7        |
| 4.5 Bulk read / write . . . . .                             | 7        |
| 4.6 ReadAnyGroup — wake on any input . . . . .              | 7        |
| 4.7 Interrupting a blocked read . . . . .                   | 8        |
| 5. Writing your first module . . . . .                      | 8        |
| 5.1 A minimal controller . . . . .                          | 8        |
| 5.2 Wiring it into the application . . . . .                | 9        |
| 6. Structuring larger applications . . . . .                | 9        |
| 6.1 Nesting ModuleGroups . . . . .                          | 9        |
| 6.2 VariableGroups inside a module . . . . .                | 10       |
| 7. Connecting modules . . . . .                             | 10       |
| 7.1 Absolute paths . . . . .                                | 10       |
| 7.2 Relative paths . . . . .                                | 10       |
| 7.3 Fan-out (one producer, many consumers) . . . . .        | 11       |
| 7.4 Triggers . . . . .                                      | 11       |
| 7.5 Connecting to hardware devices (DeviceModule) . . . . . | 12       |
| 8. The application configuration file . . . . .             | 17       |
| 8.1 XML structure . . . . .                                 | 17       |
| 8.2 Reading values in C++ . . . . .                         | 18       |
| 8.3 Reading values from scripts . . . . .                   | 19       |
| 8.4 Declaring scripted modules . . . . .                    | 19       |
| 8.5 Control system visibility . . . . .                     | 19       |
| 8.6 Complete example . . . . .                              | 19       |
| 9. Testing with TestFacility . . . . .                      | 20       |
| 9.1 Basic pattern . . . . .                                 | 20       |
| 9.2 TestFacility API summary . . . . .                      | 21       |
| 9.3 Testing array variables . . . . .                       | 22       |
| 9.4 CMakeLists for tests . . . . .                          | 22       |
| 10. Scripting modules (Lua and Python) . . . . .            | 22       |

|      |                                                              |    |
|------|--------------------------------------------------------------|----|
| 10.1 | Enabling scripting modules in the config XML . . . . .       | 22 |
| 10.2 | Module structure . . . . .                                   | 23 |
| 10.3 | Accessor types in scripts . . . . .                          | 24 |
| 10.4 | Array operations in scripts . . . . .                        | 26 |
| 10.5 | Reading config values from scripts . . . . .                 | 27 |
| 10.6 | Using VariableGroups and ModuleGroups from scripts . . . . . | 28 |
| 10.7 | Interrupting a blocked read from scripts . . . . .           | 29 |
| 10.8 | Status accessors, aggregation, and monitors (Lua) . . . . .  | 30 |
| 11.  | Useful patterns and tips . . . . .                           | 31 |
| 11.1 | Initial values before the first read . . . . .               | 31 |
| 11.2 | Data validity . . . . .                                      | 31 |
| 11.3 | Version numbers . . . . .                                    | 32 |
| 11.4 | Tags . . . . .                                               | 32 |
| 11.5 | Module lifecycle hooks . . . . .                             | 32 |
| 11.6 | Generating the variable map . . . . .                        | 32 |
| 11.7 | Testable mode without TestFacility . . . . .                 | 32 |
|      | Quick reference: variable path rules . . . . .               | 32 |
|      | Further reading . . . . .                                    | 33 |

## ChimeraTK ApplicationCore — Tutorial

This tutorial walks you through everything you need to build a real-time control system application with **ChimeraTK ApplicationCore** — from setting up a fresh CMake project to writing modules, testing them, and adding scripted logic in Lua.

---

### Table of Contents

1. [What is ApplicationCore?](#)
  2. [Project setup from scratch](#)
  3. [Core concepts](#)
  4. [Accessor types](#)
  5. [Writing your first module](#)
  6. [Structuring larger applications](#)
  7. [Connecting modules](#) — paths, fan-out, triggers, hardware devices
  8. [The application configuration file](#) — XML format, C++ API, scripted modules
  9. [Testing with TestFacility](#)
  10. [Scripting modules \(Lua and Python\)](#)
  11. [Useful patterns and tips](#)
- 

### 1. What is ApplicationCore?

ApplicationCore is a C++ framework that lets you build distributed control system applications as a network of communicating, independently-running **modules**. Each module lives in its own thread, declares typed **process variables** (inputs and outputs), and implements a `mainLoop()`. The framework:

- wires the process variables together automatically based on their paths,
- handles scheduling (a module wakes up only when new data arrives),
- propagates data-validity flags and version numbers transparently,
- publishes all variables to the control system adapter (EPICS, DOOCS, OPC-UA, ...) without any adapter-specific code in your modules.

The result is application logic that is fully decoupled from hardware details and the choice of control system middleware.

## 2. Project setup from scratch

### 2.1 Required dependencies

| Package                        | Minimum version | Notes                             |
|--------------------------------|-----------------|-----------------------------------|
| CMake                          | 3.16            |                                   |
| ChimeraTK-ApplicationCore      | any             | the framework itself              |
| ChimeraTK-ControlSystemAdapter | 02.11           | control-system back-end interface |
| ChimeraTK-DeviceAccess         | 03.18           | hardware device interface         |
| Boost                          | 1.71            | filesystem, thread, chrono, ...   |
| libxml++                       | 3.x or 5.x      | XML config file parsing           |

Optional: - **Lua** / **LuaJIT** + **sol2** — for Lua scripting modules - **pybind11** — for Python scripting modules

### 2.2 Directory layout

```

MyApp/
├─ CMakeLists.txt
├─ include/
│   └─ MyApp.h          ← Application subclass
│   └─ MyModule.h       ← ApplicationModule subclass(es)
├─ src/
│   └─ main.cc
│   └─ MyApp.cc
│   └─ MyModule.cc
├─ MyApp-config.xml     ← runtime configuration (see §8)
└─ tests/
    └─ CMakeLists.txt
    └─ testMyModule.cc

```

### 2.3 CMakeLists.txt

```

cmake_minimum_required(VERSION 3.16)
project(MyApp VERSION 1.0.0)

set(CMAKE_CXX_STANDARD 17)

```

```

find_package(ChimeraTK-ApplicationCore REQUIRED)

add_executable(MyApp
    src/main.cc
    src/MyApp.cc
    src/MyModule.cc
)

target_include_directories(MyApp PRIVATE include)
target_link_libraries(MyApp PRIVATE ChimeraTK::ApplicationCore)

```

The ChimeraTK::ApplicationCore imported target already carries all transitive dependencies (Boost, libxml++, ControlSystemAdapter, DeviceAccess).

## 2.4 main.cc

```

#include "MyApp.h"

int main(int argc, char* argv[]) {
    MyApp app("MyApp"); // name must match the config file prefix
    app.initialise();
    app.run();           // blocks until application is stopped
    return 0;
}

```

---

## 3. Core concepts

### 3.1 Class hierarchy

```

ctk::Application  (extends ModuleGroup – the root of the module tree)
├─ ctk::ModuleGroup  (container; organises modules into named sub-trees)
│   └─ ctk::VariableGroup  (organises variables within a module)
│       └─ ctk::ApplicationModule  (runs mainLoop() in its own thread)

```

### 3.2 Application

Subclass ctk::Application, declare your modules as member variables, and let the framework do the rest:

```

// include/MyApp.h
#include <ChimeraTK/ApplicationCore/Application.h>
#include "MyModule.h"

struct MyApp : public ChimeraTK::Application {
    using ChimeraTK::Application::Application;
    ~MyApp() override { shutdown(); }
}

```

```
MyModule myModule{this, "MyModule", "Does useful work"};
};
```

The destructor **must** call shutdown().

### 3.3 ApplicationModule

An ApplicationModule is the basic unit of computation. It runs its mainLoop() in a dedicated thread, and owns **accessors** (typed handles to process variables).

```
// include/MyModule.h
#include <ChimeraTK/ApplicationCore/ApplicationModule.h>
#include <ChimeraTK/ApplicationCore/ScalarAccessor.h>

namespace ctk = ChimeraTK;

struct MyModule : public ctk::ApplicationModule {
    using ctk::ApplicationModule::ApplicationModule;

    ctk::ScalarPushInput<double> measurement{this, "measurement", "V", "Voltage reading"};
    ctk::ScalarOutput<double> result {this, "result", "V", "Processed result"};

    void mainLoop() override;
};
```

### 3.4 ModuleGroup and VariableGroup

Use ModuleGroup to group related modules under a common path prefix:

```
struct SensorUnit : public ctk::ModuleGroup {
    using ctk::ModuleGroup::ModuleGroup;

    TemperatureSensor temperature{this, "Temperature", "description"};
    PressureSensor pressure {this, "Pressure", "description"};
};
```

Use VariableGroup to sub-divide the variable space *within* a module without spawning a new thread:

```
struct PIDModule : public ctk::ApplicationModule {
    struct Gains : public ctk::VariableGroup {
        using ctk::VariableGroup::VariableGroup;
        ctk::ScalarPollInput<float> kP{this, "kP", "", "Proportional gain"};
        ctk::ScalarPollInput<float> kI{this, "kI", "", "Integral gain"};
        ctk::ScalarPollInput<float> kD{this, "kD", "", "Derivative gain"};
    } gains{this, "Gains", "PID gain parameters"};

    ctk::ScalarPushInput<float> error{this, "error", "", "Control error"};
    ctk::ScalarOutput<float> output{this, "output", "", "Control output"};

    void mainLoop() override;
};
```

```
};
```

## 4. Accessor types

Accessors are the typed handles that connect module variables to the process variable network.

### 4.1 Scalar accessors

| Type                     | Direction          | Mode | When use                                             |
|--------------------------|--------------------|------|------------------------------------------------------|
| ScalarPushInput<T>       | input              | push | Block until the sender writes a new value            |
| ScalarPollInput<T>       | input              | poll | Read the latest value on demand (no blocking)        |
| ScalarOutput<T>          | output             | push | Write a value and wake up downstream modules         |
| ScalarPushInputWriteBack | input + write-back | push | Like PushInput but can also write back to the sender |
| ScalarOutputPushReadBack | output + readback  | push | Output with a readback channel                       |

Constructor signature (all scalar types):

```
ScalarPushInput<double> input{
    this,           // owner (this module or variable group)
    "myVar",        // variable name / path
    "m/s",          // engineering unit
    "description"   // human-readable description
    // optional: {"tag1", "tag2"} ← tags
};
```

### 4.2 Array accessors

Array accessors add a `nElements` parameter:

```
ArrayPushInput<int32_t> waveform{this, "waveform", "counts", 1024, "ADC waveform"};
ArrayOutput<float> spectrum{this, "spectrum", "dB", 512, "FFT spectrum"};
```

| Type              | Direction | Mode |
|-------------------|-----------|------|
| ArrayPushInput<T> | input     | push |
| ArrayPollInput<T> | input     | poll |
| ArrayOutput<T>    | output    | push |

### 4.3 Void accessors (triggers)

```
VoidInput trigger{this, "trigger", "External trigger"};
VoidOutput tick {this, "tick", "Heartbeat"};
```

No data is transferred — only the event/trigger itself.

### 4.4 Reading and writing

```
// --- Reading ---
input.read();           // blocking: wait for new data
input.readNonBlocking(); // returns true if new data was available
input.readLatest();     // discard older values, get the latest

double val = double(input); // cast to extract the value
std::vector<int32_t> v = std::vector<int32_t>(arrayInput);

// --- Writing ---
output = 3.14;           // set value (does not publish yet)
output.write();           // publish to subscribers
output.setAndWrite(3.14); // set + publish in one call
output.writeIfDifferent(val); // publish only if value changed
output.writeDestructively(); // optimised write (value may be moved)
```

### 4.5 Bulk read / write

When a module has several inputs that should all be read before processing:

```
void mainLoop() override {
    writeAll(); // write all outputs (send initial values)
    while (true) {
        readAll(); // block until ALL push inputs have new data
        // ... compute ...
        writeAll();
    }
}
```

`readAll()` blocks until every `ScalarPushInput` and `ArrayPushInput` in the module has received at least one update.

### 4.6 ReadAnyGroup — wake on any input

```
void mainLoop() override {
    ctk::ReadAnyGroup group{input1, input2, input3};

    while (true) {
        auto notification = group.readAny(); // wakes on ANY input
        // all accessors in the group have their latest value
        // notification.getId() tells which accessor triggered the wake-up
    }
}
```

```

        output.setAndWrite(double(input1) + double(input2));
    }
}

```

## 4.7 Interrupting a blocked read

`ReadAnyGroup::interrupt()` (and the same method on individual accessors) unblocks any thread currently waiting in `read()` or `readAny()` by injecting a `boost::thread_interrupted` exception into the transfer queue.

The framework calls this automatically during shutdown — your `mainLoop()` does not need to handle it. You only need `interrupt()` yourself in advanced patterns where one thread must forcibly wake another, for example a watchdog that aborts a stalled module:

```

// From a watchdog thread or another module:
group.interrupt(); // unblocks the thread blocked in group.readAny()

```

The interrupted thread receives `boost::thread_interrupted`. If you let it propagate, `ApplicationCore` treats it as a clean shutdown signal. If you catch it, always re-throw after any clean-up so the framework can still terminate the thread correctly:

```

try {
    auto id = group.readAny();
    // normal processing ...
}
catch(const boost::thread_interrupted&) {
    // optional clean-up ...
    throw; // always re-throw so the framework can shut down cleanly
}

```

---

## 5. Writing your first module

### 5.1 A minimal controller

```

// include/Controller.h
#pragma once
#include <ChimeraTK/ApplicationCore/ApplicationModule.h>
#include <ChimeraTK/ApplicationCore/ScalarAccessor.h>

namespace ctk = ChimeraTK;

struct Controller : public ctk::ApplicationModule {
    using ctk::ApplicationModule::ApplicationModule;

    ctk::ScalarPollInput<float> setpoint {this, "setpoint", "degC", "Temperature setpoint"};
    ctk::ScalarPushInput<float> readback {this, "readback", "degC", "Actual temperature"};
    ctk::ScalarOutput<float> current {this, "current", "mA", "Heater current"};
}

```



```

    void mainLoop() override;
};

// src/Controller.cc
#include "Controller.h"

void Controller::mainLoop() {
    const float gain = 100.0f;

    while (true) {
        readback.read(); // block until a new temperature arrives

        setpoint.read(); // poll: read the latest setpoint (never blocks)
        current = gain * (setpoint - readback);
        current.write();
    }
}

```

## 5.2 Wiring it into the application

```

// include/MyApp.h
#include <ChimeraTK/ApplicationCore/Application.h>
#include "Controller.h"

struct MyApp : public ChimeraTK::Application {
    using ChimeraTK::Application::Application;
    ~MyApp() override { shutdown(); }

    Controller controller{this, "Controller", "Temperature controller"};
};

```

The process variable `/Controller/setpoint` and `/Controller/readback` are now automatically connected to the control system adapter and to any other module that uses the same path.

## 6. Structuring larger applications

### 6.1 Nesting ModuleGroups

```

struct MyApp : public ctk::Application {
    using ctk::Application::Application;
    ~MyApp() override { shutdown(); }

    struct ControlUnit : public ctk::ModuleGroup {
        using ctk::ModuleGroup::ModuleGroup;
        Controller    controller    {this, "Controller",    "PID controller"};
        SafetyMonitor safety        {this, "SafetyMonitor", "Over-temperature guard"};
    } controlUnit{this, "ControlUnit", "Main control group"};
};

```

```

    DataLogger dataLogger{this, "DataLogger", "Logs data to file"};
};

```

The resulting variable paths are /ControlUnit/Controller/..., /ControlUnit/SafetyMonitor/..., and /DataLogger/...

## 6.2 VariableGroups inside a module

```

struct AdvancedModule : public ctk::ApplicationModule {
    struct Status : public ctk::VariableGroup {
        using ctk::VariableGroup::VariableGroup;
        ctk::ScalarOutput<int32_t> errorCode {this, "errorCode", "", "Last error"};
        ctk::ScalarOutput<bool> running {this, "running", "", "Is running"};
    } status{this, "Status", "Module status"};

    ctk::ScalarPushInput<double> input {this, "input", "V", "Raw signal"};
    ctk::ScalarOutput<double> output{this, "output", "V", "Processed signal"};

    void mainLoop() override;
};

```

Variable paths: /<module>/Status/errorCode, /<module>/Status/running, /<module>/input, ...

## 7. Connecting modules

Process variables connect implicitly by **path**. Two accessors that share the same absolute path form a producer-consumer pair; the framework inserts fan-outs automatically when one producer has multiple consumers.

### 7.1 Absolute paths

```

// Producer (in ModuleA):
ctk::ScalarOutput<float> out{this, "/Shared/temperature", "degC", "desc"};

// Consumer (in ModuleB):
ctk::ScalarPushInput<float> temp{this, "/Shared/temperature", "degC", "desc"};

```

Both refer to the same process variable, regardless of where in the module hierarchy they live.

### 7.2 Relative paths

Paths that do not start with / are resolved relative to the module's position in the hierarchy. Use .. to go up one level:

```

// Inside ControlUnit/SafetyMonitor, read from ControlUnit/Controller/current:
ctk::ScalarPushInput<float> current{this, "../Controller/current", "mA", "desc"};

```

### 7.3 Fan-out (one producer, many consumers)

No special code is needed. If two modules both declare a `ScalarPushInput` at the same path, the framework creates an internal fan-out so both modules receive every update independently.

### 7.4 Triggers

A **trigger** is any process variable that, when written, causes something to happen — either waking up a module or telling a `DeviceModule` to read its poll registers. Triggers carry no data; they are purely an event signal. The trigger path passed to `DeviceModule` (§7.5) can come from any of the four sources below.

**PeriodicTrigger — fixed-rate software timer** For modules or devices that must run at a fixed rate:

```
#include <ChimeraTK/ApplicationCore/PeriodicTrigger.h>

struct MyApp : public ctk::Application {
    ctk::PeriodicTrigger timer{this, "Timer", "10 Hz clock", 100 /*ms*/};
    MyModule myModule{this, "MyModule", "desc"};
};

// In MyModule – wakes every 100 ms:
ctk::VoidInput tick{this, "/Timer/tick", "Periodic trigger"};

void MyModule::mainLoop() {
    while (true) {
        tick.read();
        doWork();
    }
}
```

`PeriodicTrigger` publishes a `VoidOutput` at `/<name>/tick`. The default period (if the fourth argument is omitted) is 1000 ms.

**ApplicationModule VoidOutput — software-generated trigger** Any module can act as a trigger source by writing to a `VoidOutput`. This is useful when the trigger rate is determined by application logic rather than a clock — for example, triggering a readout after a command is received:

```
struct CommandModule : public ctk::ApplicationModule {
    using ctk::ApplicationModule::ApplicationModule;

    ctk::ScalarPushInput<int> command{this, "command", "", "Incoming command"};
    ctk::VoidOutput          doReadout{this, "doReadout", "Trigger ADC readout"};

    void mainLoop() override {
        while (true) {
            command.read();
            if (command == 1) {
```

```

        doReadout.write(); // wakes any module / DeviceModule listening on this path
    }
}
};

// DeviceModule driven by the software trigger:
ctk::DeviceModule adc{this, "MyADC", "/CommandModule/doReadout"};

```

**Device interrupt register — hardware-generated trigger** If a device has a register with push semantics (e.g. a hardware interrupt line), that register’s path can trigger reads on a *different* device. The framework reads the interrupt register as soon as the hardware fires it and then clocks the poll device:

```

// Device A has a push register /TRIG/irq that fires on hardware interrupt.
// Device B has only poll registers; use A's interrupt to clock B's reads.
ctk::DeviceModule deviceA{this, "DeviceA"}; // provides /TRIG/irq
ctk::DeviceModule deviceB{this, "DeviceB", "/TRIG/irq"}; // polled on A's interrupt

```

This requires that the interrupt register appears as a `PUSH_OUT` (or equivalent interrupt-capable) entry in the device map file for DeviceA.

**Control-system variable — operator-triggered readout** Any write arriving from the control system (EPICS, DOOCS, OPC-UA, ...) to a given path counts as a trigger. This allows an operator panel button to initiate a device readout on demand:

```

// Operator writes 1 to /Commands/readNow from the control-system panel.
// This path has no producer on the C++ side – the control system is the producer.
ctk::DeviceModule adc{this, "MyADC", "/Commands/readNow"};

```

No `VoidOutput` or `PeriodicTrigger` needs to be declared; the framework accepts a trigger from any source that writes to the configured path.

## 7.5 Connecting to hardware devices (DeviceModule)

ApplicationCore uses **ChimeraTK DeviceAccess** as its hardware abstraction layer. You never call `device.open()` or `device.getRegisterAccessor()` inside a module. Instead, `DeviceModule` publishes every register in a device’s map file as a process variable at the corresponding path — and the normal path-matching mechanism wires them to your module’s accessors automatically.

**Setting the DMAP file** A DMAP file maps device aliases to backend CDDs (connection strings). Set it once, before any `DeviceModule` is constructed. The safest place is as the first data member of your `Application`:

```

#include <ChimeraTK/ApplicationCore/DeviceModule.h>

struct MyApp : public ctk::Application {
    MyApp() : Application("MyApp") {}
    ~MyApp() override { shutdown(); }
};

```

```

    ctk::SetDMapFilePath dmap{"myapp.dmap"};    // must come before DeviceModules

    // modules and devices declared below ...
};

```

myapp.dmap is a text file with one line per device:

```

# alias          CDD (backend descriptor)
MyBoard          (pcie?device=/dev/amc_pcie_0&map=myboard.mmap)
Dummy0           (dummy?map=test.map)

```

You can also skip the DMAP and pass a CDD directly as the alias string.

```

struct MyApp : public ctk::Application {
    MyApp() : Application("MyApp") {}
    ~MyApp() override { shutdown(); }

    ctk::SetDMapFilePath dmap{"myapp.dmap"};

    MyControlModule controller{this, "Controller", "PID loop"};

    // Simplest form: expose all registers of "MyBoard" as process variables
    ctk::DeviceModule board{this, "MyBoard"};
};

```

**Declaring a DeviceModule** The DeviceModule constructor signature is:

```

DeviceModule(ModuleGroup* owner,
             const std::string& deviceAliasOrCDD,
             const std::string& triggerPath = {},           // optional
             std::function<void(Device&)> initHandler = nullptr, // optional
             const std::string& pathInDevice = "/");        // optional

```

**Wiring: path matching** A device register named /Controller/setpoint in the map file becomes a process variable at path /Controller/setpoint. An accessor in MyControlModule declared at the relative path setpoint (which the framework resolves to /Controller/setpoint) is automatically connected to that hardware register:

```

struct MyControlModule : public ctk::ApplicationModule {
    using ctk::ApplicationModule::ApplicationModule;

    // Written to device register /Controller/setpoint on every write()
    ctk::ScalarOutput<float> setpoint{this, "setpoint", "V", "DAC output"};

    // Fed from device register /Controller/readback on every new sample
    ctk::ScalarPushInput<float> readback{this, "readback", "V", "ADC input"};

    void mainLoop() override {

```

```

        while (true) {
            readback.read();
            setpoint.setAndWrite(readback * 1.1f);
        }
    }
};

```

No `connectTo()`, no explicit fan-out code — the framework does it.

**Poll registers and triggers** Some registers have no interrupt and must be read on demand. Pass the path of a trigger variable as the second argument to `DeviceModule`. Any of the four trigger sources described in §7.4 can be used — the most common is a `PeriodicTrigger`:

```

struct MyApp : public ctk::Application {
    MyApp() : Application("MyApp") {}
    ~MyApp() override { shutdown(); }

    ctk::SetDMapFilePath    dmap{"myapp.dmap"};
    ctk::PeriodicTrigger    timer{this, "Timer", "10 Hz clock", 100 /*ms*/};

    // All poll registers on MyADC are read whenever /Timer/tick fires
    ctk::DeviceModule adc{this, "MyADC", "/Timer/tick"};

    MyControlModule controller{this, "Controller", "PID loop"};
};

```

Push registers (interrupt-driven) do not need a trigger; they are read as soon as the device notifies the framework.

**Exposing only part of a device** Use `pathInDevice` to expose a sub-tree of the register map:

```

// Only registers under /DAC/CH0 in the device are visible,
// mounted at the DeviceModule's position in the hierarchy
ctk::DeviceModule dacCh0{this, "MyDAC", "", nullptr, "/DAC/CH0"};

```

**Initialisation handler** If the device needs configuration registers written after (re-)opening, pass a callback. It is called automatically every time the device comes online — including after a fault recovery:

```

ctk::DeviceModule fpga{this, "MyFPGA", "/Timer/tick",
    [](ChimeraTK::Device& dev) {
        dev.write<uint32_t>("/CTRL/reset", 1);
        dev.write<uint32_t>("/CTRL/enable", 1);
    }};

```

**Fault handling and recovery** You do not need to write any recovery code. If a `ChimeraTK::runtime_error` is thrown during a register transfer, the framework:

1. Marks all variables from that device as `DataValidity::faulty`.

2. Keeps retrying to reopen the device in the background.
3. Calls the initialisation handler again once the device is back.
4. Resumes normal operation and clears the faulty flags.

Your `mainLoop` can check `dataValidity()` for safety-critical code paths, but this is optional.

## Mental model

```
Hardware register /Controller/setpoint    (device map file)
      ‡ path matching by the framework
ApplicationModule accessor "setpoint"    (resolved to /Controller/setpoint)
      ‡
Control system variable    /Controller/setpoint    (EPICS PV, D00CS property, ...)
```

The same variable path connects hardware, application logic, and the control system — your module code never needs to know which end it is talking to.

**Reading the DMAP path and device alias from the config XML** `Application::Application()` constructs the `ConfigReader` before any derived-class members are initialized, so `getConfigReader()` is already usable in member initializers. This lets you move the DMAP path and device aliases into the XML config file instead of hard-coding them:

```
struct MyApp : public ctk::Application {
    MyApp() : Application("MyApp") {}
    ~MyApp() override { shutdown(); }

    ctk::SetDMapFilePath dmap{
        getConfigReader().get<std::string>("dmapFile", "myapp.dmap")};

    ctk::DeviceModule board{
        this,
        getConfigReader().get<std::string>("boardDevice", "MyBoard"),
        getConfigReader().get<std::string>("boardTrigger", "/Timer/tick")};

    ctk::PeriodicTrigger timer{this, "Timer", "10 Hz clock", 100};
    MyControlModule controller{this, "Controller", "PID loop"};
};
```

```
<!-- MyApp-config.xml -->
<configuration>
  <variable name="dmapFile"      type="string" value="myapp.dmap" />
  <variable name="boardDevice"    type="string" value="MyBoard" />
  <variable name="boardTrigger"   type="string" value="/Timer/tick" />
</configuration>
```

**Multiple devices** You can declare as many `DeviceModule` instances as you need. The framework manages each device's connection, fault handling, and recovery independently:

```
struct MyApp : public ctk::Application {
    MyApp() : Application("MyApp") {}
```

```

~MyApp() override { shutdown(); }

ctk::SetDMapFilePath dmap{
    getConfigReader().get<std::string>("dmapFile", "myapp.dmap")};

ctk::PeriodicTrigger timer{this, "Timer", "10 Hz clock", 100};

// Each DeviceModule is a separate, independently-managed device connection.
ctk::DeviceModule adc {this,
    getConfigReader().get<std::string>("adcDevice", "MyADC"), "/Timer/tick"};
ctk::DeviceModule dac {this,
    getConfigReader().get<std::string>("dacDevice", "MyDAC")};
ctk::DeviceModule fpga{this,
    getConfigReader().get<std::string>("fpgaDevice", "MyFPGA"), "/Timer/tick",
    [](ChimeraTK::Device& dev) {
        dev.write<uint32_t>("/CTRL/enable", 1);
    }};

MyControlModule controller{this, "Controller", "PID loop"};
MonitorModule monitor {this, "Monitor", "Alarm logic"};
};

```

```

<!-- MyApp-config.xml -->
<configuration>
  <variable name="dmapFile"      type="string" value="myapp.dmap" />
  <variable name="adcDevice"     type="string" value="MyADC" />
  <variable name="dacDevice"     type="string" value="MyDAC" />
  <variable name="fpgaDevice"    type="string" value="MyFPGA" />
</configuration>

```

```

# myapp.dmap
MyADC    (pcie?device=/dev/adc_0&map=adc.mmap)
MyDAC    (pcie?device=/dev/dac_0&map=dac.mmap)
MyFPGA   (pcie?device=/dev/fpga_0&map=fpga.mmap)

```

If the ADC faults, the DAC and FPGA keep running — fault isolation is per `DeviceModule`.

**Splitting one physical device into multiple `DeviceModules`** Sometimes one physical device serves logically independent subsystems with different triggers or ownership. Use `pathInDevice` to expose sub-trees, and pass the **same CDD string** (not different alias names) so the framework reuses a single backend connection:

```

<!-- MyApp-config.xml -->
<configuration>
  <variable name="dmapFile"      type="string" value="myapp.dmap" />
  <variable name="fpgaCDD"       type="string" value="(pcie?device=/dev/amc_pcie_0&map=fpga.mmap)" />
</configuration>

```

```

# myapp.dmap

```



```
MyFPGA    (pcie?device=/dev/amc_pcie_0&map=fpga.mmap)

struct MyApp : public ctk::Application {
    MyApp() : Application("MyApp") {}
    ~MyApp() override { shutdown(); }

    ctk::SetDMapFilePath dmap{
        getConfigReader().get<std::string>("dmapFile", "myapp.dmap");

    ctk::PeriodicTrigger timer{this, "Timer", "10 Hz clock", 100};

    // Use the CDD (not the alias) so the framework shares one backend instance.
    // The CDD is read from XML so it can be changed without recompilation.
    ctk::DeviceModule fpgaDSP{this,
        getConfigReader().get<std::string>("fpgaCDD"), "/Timer/tick", nullptr, "/DSP"};
    ctk::DeviceModule fpgaIO {this,
        getConfigReader().get<std::string>("fpgaCDD"), "/Timer/tick", nullptr, "/IO"};
};
```

Registers under /DSP/... and /IO/... are published independently, but share one backend instance. Note that & inside an XML attribute value must be written as &amp;.

## 8. The application configuration file

The configuration file is named <AppName>-config.xml and must reside in the working directory from which the application is launched. It is loaded automatically at startup.

MyApp-config.xml ← loaded when Application("MyApp") is constructed

### 8.1 XML structure

The document element must be <configuration>:

```
<configuration>
  <!-- variables and modules go here -->
</configuration>
```

```
<variable name="myValue" type="int32" value="42" />
```

#### Scalar variables

| Attribute | Required | Description                    |
|-----------|----------|--------------------------------|
| name      | yes      | Variable name (path component) |
| type      | yes      | Data type (see table below)    |
| value     | yes      | The scalar value as a string   |

## Supported types

| XML type | C++ type    | Notes                 |
|----------|-------------|-----------------------|
| int8     | int8_t      |                       |
| uint8    | uint8_t     |                       |
| int16    | int16_t     |                       |
| uint16   | uint16_t    |                       |
| int32    | int32_t     |                       |
| uint32   | uint32_t    |                       |
| int64    | int64_t     |                       |
| uint64   | uint64_t    |                       |
| float    | float       | Also usable as double |
| double   | double      |                       |
| string   | std::string |                       |
| boolean  | bool        | Values: true / false  |

**Array variables** Omit the value attribute and provide <value> child elements, one per element:

```
<variable name="myArray" type="float">
  <value i="0" v="1.0" />
  <value i="1" v="2.5" />
  <value i="2" v="3.14" />
</variable>
```

| Attribute | Description      |
|-----------|------------------|
| i         | Zero-based index |
| v         | Element value    |

**Hierarchical modules (namespacing)** Use <module name="..."> to group variables under a path prefix. Nesting is unlimited:

```
<module name="Sensors">
  <variable name="calibration" type="float" value="1.05" />
  <module name="Channel1">
    <variable name="offset" type="int32" value="10" />
  </module>
</module>
```

Variables are accessed with paths Sensors/calibration, Sensors/Channel1/offset, etc.

## 8.2 Reading values in C++

Include <ChimeraTK/ApplicationCore/ConfigReader.h>, or use the appConfig() free function available inside any ApplicationModule::mainLoop():

```
auto& cfg = appConfig();

// Scalar
```

```

float gain = cfg.get<float>("gain");

// With a default (returned when the key is absent)
int32_t cycles = cfg.get<int32_t>("maxCycles", 100);

// Array
std::vector<float> lut = cfg.get<std::vector<float>>("lookupTable");

// List sub-module names at a given path
std::vector<std::string> channels = cfg.getModules("Sensors");
// → {"Channell"}

```

Path rules: separator is /; a leading / is stripped automatically so `/foo` and `foo` are equivalent. Names are case-sensitive.

### 8.3 Reading values from scripts

See §10.5 for the Lua and Python config APIs. Lua scripts can use `xmlConfig.get("path", default)` (type inferred from the default) or the legacy `cfg:get(DataType.T, "path")` form; Python uses `cfg.get(DataType.T, "path")`.

### 8.4 Declaring scripted modules

The `<LuaModules>` and `<PythonModules>` sections tell the framework which scripts to load. Each `<module>` child specifies one script; see §10.1 for full details.

```

<module name="LuaModules">
  <module name="Control">
    <variable name="path" type="string" value="control" />
  </module>
</module>

```

### 8.5 Control system visibility

All variables declared in the config file are automatically published as process variables under `/Configurable/` in the control system hierarchy. Operators can read (and, for writable variables, modify) them at run time through the standard control-system interface.

### 8.6 Complete example

```

<configuration>

  <!-- Scripted modules -->
  <module name="LuaModules">
    <module name="Control">
      <variable name="path" type="string" value="control" />
    </module>
  </module>

```

```

<!-- Scalars at the top level -->
<variable name="maxCurrent" type="float" value="5.0" />
<variable name="deviceName" type="string" value="DEV001" />
<variable name="enableDebug" type="boolean" value="false" />

<!-- Nested namespace -->
<module name="PID">
  <variable name="kP" type="double" value="1.2" />
  <variable name="kI" type="double" value="0.05" />
  <variable name="kD" type="double" value="0.01" />
</module>

<!-- Array -->
<variable name="lookupTable" type="float">
  <value i="0" v="0.0" />
  <value i="1" v="0.25" />
  <value i="2" v="0.5" />
  <value i="3" v="0.75" />
  <value i="4" v="1.0" />
</variable>

</configuration>

```

Corresponding C++ reads:

```

auto& cfg = appConfig();
float maxI = cfg.get<float>("maxCurrent");           // 5.0
double kP  = cfg.get<double>("PID/kP");              // 1.2
auto lut   = cfg.get<std::vector<float>>("lookupTable"); // 5-element vector

```

## 9. Testing with TestFacility

TestFacility runs the application in a deterministic, single-step mode — perfect for unit tests with Boost.Test.

### 9.1 Basic pattern

```

#define BOOST_TEST_MODULE testController
#include <boost/test/included/unit_test.hpp>
#include <ChimeraTK/ApplicationCore/TestFacility.h>
#include "MyApp.h"

namespace ctk = ChimeraTK;

BOOST_AUTO_TEST_CASE(testBasicControl) {
  MyApp app("MyApp");
  ctk::TestFacility tf(app);
}

```

```

// Obtain accessors for test-side reads and writes
auto setpoint = tf.getScalar<float>("/Controller/setpoint");
auto readback = tf.getScalar<float>("/Controller/readback");
auto current  = tf.getScalar<float>("/Controller/current");

// Provide initial values before the threads start
tf.setScalarDefault<float>("/Controller/setpoint", 100.0f);

// Start all threads; modules block at their first blocking read
tf.runApplication();

// Provide readback and step the application
readback = 80.0f;
readback.write();
tf.stepApplication();

// Read the result
BOOST_TEST(current.readNonBlocking());
BOOST_TEST(float(current) == 100.0f * (100.0f - 80.0f),
            boost::test_tools::tolerance(0.01f));
}

```

## 9.2 TestFacility API summary

```

ctk::TestFacility tf(app);

// --- Setup (before runApplication) ---
tf.setScalarDefault<T>("/path", value);
tf.setArrayDefault<T>("/path", {v0, v1, v2});

// --- Start ---
tf.runApplication();

// --- Control ---
tf.stepApplication();           // run one full iteration of all modules

// --- Low-level accessors ---
auto s = tf.getScalar<T>("/path"); // ScalarRegisterAccessor<T>
auto a = tf.getArray<T>("/path");  // OneDRegisterAccessor<T>

// Using the low-level accessor:
s = 42;
s.write();           // push to application
tf.stepApplication();
BOOST_TEST(s.readNonBlocking());
BOOST_TEST(float(s) == expected);

```

### 9.3 Testing array variables

```
auto arrayIn = tf.getArray<int32_t>("/SomeName/ArrayIn");
auto arrayOut = tf.getArray<int32_t>("/SomeName/ArrayOut");

arrayIn = {10, 20, 30};
arrayIn.write();
tf.stepApplication();

BOOST_TEST(arrayOut.readNonBlocking());
std::vector<int32_t> expected{60, 61, 62, /*...*/};
BOOST_TEST(std::vector<int32_t>(arrayOut) == expected,
            boost::test_tools::per_element());
```

### 9.4 CMakeLists for tests

```
find_package(Boost COMPONENTS unit_test_framework REQUIRED)
find_package(ChimeraTK-ApplicationCore REQUIRED)

add_executable(testMyModule tests/testMyModule.cc src/MyModule.cc)
target_include_directories(testMyModule PRIVATE include)
target_link_libraries(testMyModule PRIVATE
    ChimeraTK::ApplicationCore
    Boost::unit_test_framework
)
add_test(NAME testMyModule COMMAND testMyModule)
```

---

## 10. Scripting modules (Lua and Python)

ApplicationCore supports two scripting back-ends for writing modules without recompilation:

| Feature        | Lua                            | Python                      |
|----------------|--------------------------------|-----------------------------|
| Build flag     | ENABLE_LUA_BINDINGS=ON         | ENABLE_PYTHON_BINDINGS=ON   |
| Config section | <LuaModules>                   | <PythonModules>             |
| Style          | Functional (closure or method) | Class-based (subclass)      |
| Arrays         | 1-based, plain tables          | 0-based, numpy arrays       |
| Interrupts     | String sentinel                | ThreadInterrupted exception |
| Config API     | xmlConfig.get(path, default)   | cfg.get(DataType.T, path)   |

The sections below show both languages side by side. The C++ API (§3–§5) is the authoritative reference for behaviour; the scripting bindings mirror it closely.

#### 10.1 Enabling scripting modules in the config XML

**Lua** — add a <LuaModules> section. Each <module> child specifies one Lua module name (passed to require, no .lua extension — identical convention to Python):

```

<configuration>
  <module name="LuaModules">
    <module name="Controller">
      <variable name="path" type="string" value="controller" />
    </module>
  </module>

  <!-- any other config variables are accessible via xmlConfig.get() or appConfig() -->
  <variable name="gain" type="float" value="2.5" />
</configuration>

```

The path is a Lua module name resolved via `package.path` (the working directory is searched first).

**Python equivalent** — use `<PythonModules>` instead. The path value is a Python module name (passed to `import`), following the same convention:

```

<configuration>
  <module name="PythonModules">
    <module name="Controller">
      <variable name="path" type="string" value="controller" />
    </module>
  </module>

  <variable name="gain" type="float" value="2.5" />
</configuration>

```

The module is imported from Python's `sys.path`, so `controller.py` must be importable from the working directory (or installed as a package).

## 10.2 Module structure

**Lua** — there are two equivalent styles. The preferred *method-call* style assigns `mainLoop` as a function and uses method syntax for accessor factories:

```

-- controller.lua (preferred style)

local mod = ApplicationModule(app, "Controller", "Lua PID controller")

-- Declare accessors using method syntax on the module object.
mod.setpoint = mod:ScalarPollInput(DataType.float32, "setpoint", "degC", "Setpoint")
mod.readback = mod:ScalarPushInput(DataType.float32, "readback", "degC", "Actual temperature")
mod.output   = mod:ScalarOutput   (DataType.float32, "output",   "mA",   "Heater current")
mod.gain      = mod:ScalarPollInput(DataType.float32, "gain",      "",     "Control gain")

-- Read a config value – type is inferred from the Lua default.
local scale = xmlConfig.get("Controller/scale", 1.0) -- number → float64

function mod:mainLoop()
  -- File-scope locals (scale) and accessors are available as upvalues.
  self.output:setAndWrite(0.0)
end

```

```

while true do
    local sp = self.setpoint:readAndGet()
    local rb = self.readback:readAndGet()
    self.output:setAndWrite(scale * self.gain * (sp - rb))
end
end

```

The *closure* style (passing `mainLoop` as the third argument to `ApplicationModule`) is also supported for backward compatibility. File-scope locals — including accessor objects — are migrated into each module's own Lua VM as upvalues automatically, so they are always available inside `mainLoop` regardless of which style you use.

**Python equivalent** — subclass `ac.ApplicationModule` and override `mainLoop`. Accessors are created in `__init__` and the instance is registered on `ac.app`:

```

# controller.py
import PyApplicationCore as ac

class Controller(ac.ApplicationModule):

    def __init__(self, owner, name, description):
        super().__init__(owner, name, description)
        # Declare accessors in __init__ - they must exist before threads start
        self.setpoint = ac.ScalarPollInput(ac.DataType.float32, self, "setpoint", "degC", "Setpoint")
        self.readback = ac.ScalarPushInput(ac.DataType.float32, self, "readback", "degC", "Actual temperature")
        self.output = ac.ScalarOutput(ac.DataType.float32, self, "output", "mA", "Heater current")
        self.gain = ac.ScalarPollInput(ac.DataType.float32, self, "gain", "", "Control gain")

    def mainLoop(self):
        self.output.setAndWrite(0.0) # publish initial value

        while True:
            sp = self.setpoint.readAndGet()
            rb = self.readback.readAndGet()
            self.output.setAndWrite(self.gain.get() * (sp - rb))

# Register the module on the application object
ac.app.controller = Controller(ac.app, "Controller", "Python PID controller")

```

Key differences from Lua: - Accessors go in `__init__`, not after the function. - `self.gain.get()` is needed because `ScalarPollInput` does not block; calling `readAndGet()` on a poll input would also work. - The module is instantiated and attached to `ac.app` at module-import time (equivalent to the Lua `ApplicationModule(app, ...)` call).

### 10.3 Accessor types in scripts

**Lua** — preferred method-call style (module/group as `self`):



```

-- Scalars
mod:ScalarPushInput (DataType.T, "name", "unit", "desc")
mod:ScalarPollInput (DataType.T, "name", "unit", "desc")
mod:ScalarOutput    (DataType.T, "name", "unit", "desc")

-- Arrays (add element count)
mod:ArrayPushInput  (DataType.T, "name", "unit", N, "desc")
mod:ArrayPollInput  (DataType.T, "name", "unit", N, "desc")
mod:ArrayOutput     (DataType.T, "name", "unit", N, "desc")

-- Triggers (no data)
mod:VoidInput       ("name", "desc")
mod:VoidOutput      ("name", "desc")

-- Status (integer with OK/WARNING/FAULT/OFF semantics)
mod:StatusOutput    ("name", "desc")
mod:StatusPushInput ("name", "desc")
mod:StatusPollInput ("name", "desc")

```

The same methods are available on `VariableGroup` objects. The legacy free-function form (e.g., `ScalarPushInput(DataType.T, mod, "name", "unit", "desc")`) is still accepted.

`DataType` constants:

```

DataType.int8    DataType.uint8
DataType.int16   DataType.uint16
DataType.int32   DataType.uint32
DataType.int64   DataType.uint64
DataType.float32 DataType.float64
DataType.string  DataType.Boolean  DataType.Void

```

Status constants (for status accessors):

```
Status.OK    Status.WARNING    Status.FAULT    Status.OFF
```

Scalar accessors also support arithmetic operators directly, so you can write expressions without calling `:get()`:

```

self.sum:setAndWrite(self.a + self.b)      -- addition
self.out:setAndWrite(self.input * 2.5)     -- mixed accessor/number
local clamped = self.value < self.limit    -- comparison (returns boolean)

```

**Python equivalent** — identical constructor signatures; the only difference is the `ac.` prefix and `ac.DataType.*`:

```

import PyApplicationCore as ac

# Scalars
ac.ScalarPushInput (ac.DataType.T, module, "name", "unit", "desc")
ac.ScalarPollInput (ac.DataType.T, module, "name", "unit", "desc")
ac.ScalarOutput    (ac.DataType.T, module, "name", "unit", "desc")

```

```

# Arrays (add element count)
ac.ArrayPushInput  (ac.DataType.T, module, "name", "unit", N, "desc")
ac.ArrayPollInput  (ac.DataType.T, module, "name", "unit", N, "desc")
ac.ArrayOutput      (ac.DataType.T, module, "name", "unit", N, "desc")

# Triggers (no data)
ac.VoidInput        (module, "name", "desc")
ac.VoidOutput        (module, "name", "desc")

```

DataType constants (same names, ac.DataType. prefix):

```

ac.DataType.int8      ac.DataType.uint8
ac.DataType.int16     ac.DataType.uint16
ac.DataType.int32     ac.DataType.uint32
ac.DataType.int64     ac.DataType.uint64
ac.DataType.float32   ac.DataType.float64
ac.DataType.string    ac.DataType.Boolean  ac.DataType.Void

```

## 10.4 Array operations in scripts

**Lua** — 1-based indexing, plain table iteration:

```

-- Length and indexing (1-based)
local n = #self.array      -- number of elements
local v = self.array[1]    -- read element 1
self.array[2] = 42         -- write element 2 (modifies local buffer)
self.array:write()         -- push buffer to subscribers

-- readAndGet() returns the array directly; iterate with pairs
for idx, val in pairs(self.array:readAndGet()) do
    print(idx, val)
end

-- or with a numeric for
self.array:read()
for i = 1, #self.array do
    self.out[i] = self.array[i] * 2
end
self.out:write()

```

**Python equivalent** — array accessors expose the **numpy buffer protocol**, so they behave like numpy arrays (0-based, vectorised operations, **shape**, etc.):

```

# Length and indexing (0-based)
n = len(self.array)      # number of elements
v = self.array[0]        # read element 0
self.array[1] = 42       # write element 1 (modifies local buffer)
self.array.write()       # push buffer to subscribers

```

```

# readAndGet() returns the array as a numpy view; iterate normally
for val in self.array.readAndGet():
    print(val)

# vectorised operations (no explicit loop needed)
self.array.read()
self.out.set(self.array.get() * 2)  # element-wise multiply via numpy
self.out.write()

# or use numpy directly
import numpy as np
self.out.setAndWrite(np.array(self.array) * 2)

```

## 10.5 Reading config values from scripts

**Lua** — the preferred API uses the global `xmlConfig` table; the type is inferred from the Lua default value:

```

-- Type inferred from default: number → float64, string → string, boolean → Boolean
local gain    = xmlConfig.get("gain", 1.0)           -- float64
local label   = xmlConfig.get("label", "default")    -- string
local enabled = xmlConfig.get("enabled", true)       -- Boolean

-- Array: type inferred from first element of the default table
local lut     = xmlConfig.getArray("lookupTable")    -- table of numbers (no default)
local flags   = xmlConfig.getArray("flags", {false}) -- table of booleans

-- Sub-module names (for dynamic channel creation)
local modules = xmlConfig.getModules("Sensors")

-- Paths are relative to the config root (same as appConfig)
local freq = xmlConfig.get("Controller/sampleFreq", 100.0)

```

The legacy `appConfig()` form is still available for scripts that need to pass an explicit `DataType`:

```

local cfg = appConfig()
local gain    = cfg.get(DataType.float64, "gain")
local label   = cfg.get(DataType.string, "label", "default")
local table   = cfg.getArray(DataType.float64, "lookupTable")
local modules = cfg.getModules("Sensors")

```

**Python equivalent** — identical API; call `self.appConfig()` inside a module, or the global `ac.appConfig()` at script level:

```

# inside mainLoop or __init__:
cfg = self.appConfig()
gain    = cfg.get(ac.DataType.float64, "gain")
label   = cfg.get(ac.DataType.string, "label", "default")  # third arg = default
table   = cfg.getArray(ac.DataType.float64, "lookupTable")
modules = cfg.getModules("Sensors")

```

```
# at script (module-import) level:
cfg = ac.appConfig()
n_tickers = cfg.get(ac.DataType.uint32, "numberOfTickers", 0)
```

## 10.6 Using VariableGroups and ModuleGroups from scripts

**Lua:**

```
-- ModuleGroup can be created without an explicit owner (attaches to app root)
local group = ModuleGroup("Sensors", "Sensor group")
local mod = ApplicationModule(group, "Processor", "Processes sensor data")

-- VariableGroup for hierarchical variable naming
local vg = VariableGroup(mod, "Gains", "PID gain inputs")
mod.kP = vg:ScalarPollInput(DataType.float32, "kP", "", "Proportional gain")
mod.kI = vg:ScalarPollInput(DataType.float32, "kI", "", "Integral gain")

mod.raw = mod:ScalarPushInput(DataType.float32, "raw", "V", "Raw signal")
mod.processed = mod:ScalarOutput(DataType.float32, "processed", "V", "Scaled signal")

function mod:mainLoop()
    while true do
        self.raw:read()
        self.processed:setAndWrite(self.raw + 0.0)
    end
end
```

**Python equivalent** — ModuleGroup is used as an owner; VariableGroup can be subclassed or used with dynamic attribute assignment:

```
import PyApplicationCore as ac

# --- ModuleGroup ---
sensors = ac.ModuleGroup(ac.app, "Sensors", "Sensor group")

class Processor(ac.ApplicationModule):
    def __init__(self, owner, name, description):
        super().__init__(owner, name, description)
        self.raw = ac.ScalarPushInput(ac.DataType.float32, self, "raw", "V", "Raw signal")
        self.processed = ac.ScalarOutput(ac.DataType.float32, self, "processed", "V", "Scaled signal")

    def mainLoop(self):
        while True:
            self.raw.read()
            self.processed.setAndWrite(self.raw.get() * 2)

ac.app.sensors.processor = Processor(sensors, "Processor", "Processes sensor data")
```

```

# --- VariableGroup (dynamic attributes) ---
class MyMod(ac.ApplicationModule):
    def __init__(self, owner, name, description):
        super().__init__(owner, name, description)
        # inline VariableGroup using dynamic attribute assignment
        self.gains = ac.VariableGroup(self, "Gains", "PID gains")
        self.gains.kP = ac.ScalarPollInput(ac.DataType.float32, self.gains, "kP", "", "Proportional gain")
        self.gains.kI = ac.ScalarPollInput(ac.DataType.float32, self.gains, "kI", "", "Integral gain")
        self.output = ac.ScalarOutput(ac.DataType.float32, self, "output", "", "Control output")

    def mainLoop(self):
        while True:
            self.output.setAndWrite(self.gains.kP.get()) # simplified

# --- VariableGroup (subclassed) ---
class MyMod2(ac.ApplicationModule):
    class Gains(ac.VariableGroup):
        def __init__(self, owner, name, description):
            super().__init__(owner, name, description)
            self.kP = ac.ScalarPollInput(ac.DataType.float32, self, "kP", "", "Proportional gain")

    def __init__(self, owner, name, description):
        super().__init__(owner, name, description)
        self.gains = MyMod2.Gains(self, "Gains", "PID gains")

```

## 10.7 Interrupting a blocked read from scripts

**Lua** — `ReadAnyGroup` exposes an `interrupt()` method with the same semantics as the C++ version (see §4.7). When it fires, the Lua binding re-raises the interrupt as the string `"ChimeraTK::ThreadInterrupted"`. The C++ wrapper recognises this and exits cleanly, so no extra code is needed in the normal case.

If you call `interrupt()` yourself and need to distinguish it from a real error, use `pcall`:

```

local group = ReadAnyGroup()
group:add(self.input1)
group:add(self.input2)
group:finalise()

while true do
    local ok, id = pcall(function() return group:readAny() end)
    if not ok then
        if type(id) == "string" and id:find("ChimeraTK::ThreadInterrupted") then
            return -- clean shutdown; let the framework take over
        end
        error(id) -- unexpected error: propagate
    end
    -- normal processing ...
end

```

```
end
```

If you do not use `pcall`, the exception propagates automatically and the module exits cleanly — no extra code needed.

**Python equivalent** — the interrupt surfaces as a `ThreadInterrupted` exception (a real Python exception class, not a string). The framework's `mainLoopWrapper` catches it automatically, so in the common case your `mainLoop` requires no special handling.

If you call `interrupt()` yourself and need to handle it explicitly:

```
import PyApplicationCore as ac

def mainLoop(self):
    group = ac.ReadAnyGroup()
    group.add(self.input1)
    group.add(self.input2)
    group.finalise()

    while True:
        try:
            id = group.readAny()
        except ThreadInterrupted:
            return # clean shutdown; let the framework take over
            # normal processing ...
```

`ThreadInterrupted` is registered in the `__main__` namespace by the framework at startup, so it is always available without an import.

## 10.8 Status accessors, aggregation, and monitors (Lua)

Status accessors carry a four-level severity (`OFF`, `OK`, `WARNING`, `FAULT`) and integrate with Application-Core's status-propagation infrastructure.

```
local mod = ApplicationModule(app, "Controller", "PID controller")

-- Output publishes the module's health
mod.status = mod:StatusOutput("status", "Controller health")

function mod:mainLoop()
    self.status:setAndWrite(Status.OK)
    while true do
        -- ... normal processing ...
        if someError then
            self.status:setAndWrite(Status.FAULT)
        end
    end
end
```

**StatusAggregator** rolls up all `StatusOutput` variables under a `ModuleGroup` into a single output:

```
-- Aggregate every StatusOutput under 'app' into /status
StatusAggregator(app, "/status", "Overall system status", PriorityMode.fwok)
```

Priority modes: PriorityMode.fwok, fwko, fw\_warn\_mixed, ofwk.

**PeriodicTrigger** creates a regular time base (period in milliseconds):

```
PeriodicTrigger(app, "Ticker", "500 ms heartbeat", 500)
```

**Monitor utilities** watch a scalar variable and publish a status based on threshold checks:

```
-- Fault if value exceeds a threshold
MaxMonitor  (DataType.float32, app, "/Sensors/temperature", "/status", "/params/tempMax", "Temperature
MinMonitor  (DataType.float32, app, "/Sensors/flow",          "/status", "/params/flowMin",    "Flow monito
RangeMonitor(DataType.float32, app, "/Sensors/pressure",      "/status", "/params/pressure",   "Pressure mo
ExactMonitor(DataType.int32,   app, "/Control/mode",          "/status", "/params/targetMode", "Mode monit
```

## 11. Useful patterns and tips

### 11.1 Initial values before the first read

The framework guarantees that every input has a valid (possibly zero/default) value before `mainLoop()` is called. Use `setScalarDefault` / `setArrayDefault` in `TestFacility`, or the `<variable>` entries in the config file, to pre-set values.

If your module needs to publish an initial output *before* entering the blocking loop, do it at the top of `mainLoop()`:

```
void MyModule::mainLoop() {
    output.setAndWrite(0.0); // publish a sane initial value

    while (true) {
        input.read();
        output.setAndWrite(double(input) * gain);
    }
}
```

### 11.2 Data validity

The framework propagates `DataValidity::faulty` whenever a connected device reports an error. Check it in safety-critical code:

```
if (input.dataValidity() != ctk::DataValidity::ok) {
    // take safe action
    output.setAndWrite(0.0);
    continue;
}
```

### 11.3 Version numbers

Every write carries a `VersionNumber`. Push reads propagate the incoming version to subsequent writes automatically — this is how the framework detects causally-related updates and avoids double-processing. You rarely need to interact with version numbers directly.

### 11.4 Tags

Tags are arbitrary strings you can attach to accessors or modules. They are used by the framework's model layer (for filtering, generating documentation, etc.):

```
ctk::ScalarOutput<float> out{this, "out", "V", "desc", {"physicalOutput", "safety"}};
```

### 11.5 Module lifecycle hooks

Override these in your `ApplicationModule` subclass if you need finer control:

| Method                       | When called                            | Typical use              |
|------------------------------|----------------------------------------|--------------------------|
| <code>postConstruct()</code> | after construction, single-threaded    | final wiring checks      |
| <code>prepare()</code>       | before threads start, single-threaded  | write first-time outputs |
| <code>mainLoop()</code>      | in the module's own thread             | all normal processing    |
| <code>terminate()</code>     | when <code>shutdown()</code> is called | clean-up                 |

### 11.6 Generating the variable map

Call `app.generateXML("myapp.xml")` or `app.generateDOT("myapp.dot")` after construction to dump the full variable network. The XML can be loaded by control-system tools to auto-generate operator panels.

### 11.7 Testable mode without `TestFacility`

For standalone programs that need deterministic stepping:

```
app.enableTestableMode();
app.initialise();
app.run();
// ... then step manually via the internal testable-mode interface
```

This is normally only needed in integration test harnesses; regular unit tests should use `TestFacility`.

---

### Quick reference: variable path rules

| Path form                 | Meaning                                                             |
|---------------------------|---------------------------------------------------------------------|
| <code>/Foo/Bar/var</code> | Absolute path; every accessor with this path is the same variable   |
| <code>var</code>          | Relative path; resolved from the module's position in the hierarchy |



| Path form                   | Meaning                                      |
|-----------------------------|----------------------------------------------|
| <code>../Sibling/var</code> | One level up, then into <code>Sibling</code> |
| <code>SubGroup/var</code>   | One level down into <code>SubGroup</code>    |

Leading `/` is always stripped before comparison, so `/Foo` and `Foo` at the root level refer to the same variable.

---

## Further reading

- [Lua/lua.md](#) — Complete Lua scripting API reference
- [example/](#) — Full working example application (oven temperature controller)
- [tests/executables\\_src/](#) — Comprehensive test suite showing real usage patterns